

ACID Transactions

Databases - ACID Transactions



ASHISH PRATAP SINGH

SEP 11, 2024



8



Share

Imagine a banking system where a failed transfer still deducts money from one account without crediting another. Such errors can be catastrophic.

This is where ACID transactions come into play.

ACID, an acronym for Atomicity, Consistency, Isolation, and Durability, defines a set of properties that ensure database transactions are processed reliably.

In this article, we will explore each of these properties and their importance.

1. Atomicity

Atomicity ensures that a transaction is treated as an indivisible unit of work. It means that either all the operations within a transaction are successfully completed, or none of them are applied to the database.

If any part of the transaction fails, the entire transaction is rolled back, and the database is restored to its previous consistent state.

Atomicity prevents partial updates and ensures that the database remains in a valid state, even if failures occur during the transaction.

For instance, in a banking system, if you transfer money from one account to another, atomicity ensures that either both the debit and credit operations occur, or neither does.

Example:

```
BEGIN TRANSACTION;  
UPDATE accounts SET balance = balance - 100 WHERE account_id = 1;  
UPDATE accounts SET balance = balance + 100 WHERE account_id = 2;  
COMMIT;
```

If the second update fails, atomicity ensures that the first update is rolled back, maintaining consistency.

2. Consistency

Consistency ensures that a transaction brings the database from one valid state to another valid state.

It means that all the data integrity constraints, such as unique constraints, foreign key constraints, and check constraints, are satisfied before and after the transaction.

If a transaction violates any rules, it will not be committed, and the database will revert to its previous state.

Example: Consider a database with a constraint that account balances cannot be negative. A transaction attempting to withdraw more money than available will fail, preserving the consistency of the database.

3. Isolation

Isolation ensures that transactions are executed in a way that they do not interfere with each other. It means that the intermediate state of a transaction is not visible to other transactions until it is committed.

Isolation prevents data inconsistencies that can arise when multiple transactions access and modify the same data simultaneously.

Isolation Levels:

- **Read Uncommitted:** Transactions can see uncommitted changes from other transactions.
- **Read Committed:** Transactions can only see committed changes from other transactions.
- **Repeatable Read:** Ensures that if a transaction reads a row, it will read the same row consistently during its entire execution.
- **Serializable:** Provides the highest level of isolation, ensuring transactions are executed serially.

Example: In a scenario where two transactions are updating the same record, isolation ensures that one transaction's updates are not visible to the other until they are committed.

4. Durability

Durability ensures that once a transaction is committed, it will remain so, even in the event of a system failure. This means that committed data will not be lost.

This property guarantees that the results of a transaction are permanent and survive subsequent system failures, providing reliability.

Example: After a successful bank transaction, the updated balances should be stored permanently. Even if the system crashes immediately after the transaction, the committed changes will not be lost.

How do ACID Transactions Work?

Here's an example of how ACID transactions work:

1. **Begin Transaction:** A user initiates a transaction, such as transferring money from one account to another.
2. **Execute Transaction:** The database executes the transaction, performing the necessary operations.
3. **Commit Transaction:** If the transaction is successful, the changes are committed to the database.

4. **Rollback Transaction:** If an error occurs during the transaction, the database rolls back to its previous state, as if the transaction never occurred.

Databases implement ACID transactions through a combination of techniques, including:

- **Logging:** Detailed records of all transactions are kept, allowing for recovery in case of a failure.
- **Locking:** Data is locked during a transaction to prevent concurrent access and ensure isolation.
- **Two-Phase Commit:** A protocol used to coordinate the commitment of a transaction across multiple systems.

Why are ACID Transactions Important?

ACID transactions offer several benefits, including:

- **Data Integrity:** Ensures that data is consistent and accurate.
- **Reliability:** Guarantees that transactions are processed reliably, even in the face of errors.
- **Security:** Prevents data corruption and ensures that transactions are executed securely.
- **Performance:** Improves performance by allowing concurrent transactions to occur without interference.

Practical Implementation of ACID Properties

Modern relational database management systems (RDBMS) like MySQL, PostgreSQL, and Oracle implement ACID properties to ensure data integrity and reliability.

Here's how these properties manifest in practice:

- **Transaction Management:** Most databases provide SQL commands like BEGIN, COMMIT, and ROLLBACK to manage transactions.
- **Locking Mechanisms:** Databases use locks to manage concurrent transactions, ensuring isolation.
- **Write-Ahead Logging (WAL):** Techniques like WAL ensure durability by writing changes to a log before applying them to the database.
- **Constraint Enforcement:** Databases enforce constraints to maintain consistency automatically.

ACID in NoSQL Databases

While ACID properties are foundational to RDBMS, NoSQL databases like MongoDB and Cassandra often sacrifice some ACID properties for performance and scalability. However, many NoSQL databases provide configurable levels of ACID compliance or specific ACID-compliant features.

Real-World Applications of ACID Transactions

ACID transactions are used in various real-world applications, including:

- **Banking Systems:** Ensures that financial transactions are secure and reliable.
- **E-commerce Platforms:** Guarantees that orders are processed accurately and securely.
- **Healthcare Systems:** Ensures that patient data is accurate and secure.

ACID transactions are a fundamental concept in database management systems that ensure data integrity, reliability, and consistency.

By understanding and leveraging ACID transactions, developers can create database systems that are reliable, consistent, and capable of handling concurrent access and failures.

Thank you so much for reading.

If you found it valuable, hit a like ❤️ and consider subscribing for more such content every week.

If you have any questions or suggestions, leave a comment.

This post is public so feel free to share it.

Subscribe for free to receive new posts and
support my work.

Subscribe

Follow my work on other platforms:

[LinkedIn](#) | [Youtube](#) | [X](#) | [Github](#)

I hope you have a lovely day!

See you soon,

Ashish

References:

- <https://redis.io/glossary/acid-transactions/>



8 Likes

© 2024 Ashish Pratap Singh · [Privacy](#) · [Terms](#) · [Collection notice](#)
[Substack](#) is the home for great culture